

MissingFind

카메라 기반 물체 위치 추적 시스템

GitHub Copilot 구현 명세서 & 프롬프트 가이드

v1.0 | 2026.03 | 로컬 엣지 컴퓨팅 환경

1. 프로젝트 개요 (Project Overview)

MissingFind는 YOLO-World + CLIP을 활용한 로컬 엣지 AI 물체 위치 추적 시스템입니다. 별도의 태그 없이 자연어 질문만으로 분실물을 찾아주며, 모든 연산은 가정용 PC에서 오프라인으로 처리됩니다.

항목	내용
프로젝트명	MissingFind – 카메라 기반 물체 위치 추적 시스템
목표	자연어 질의 → 실내 분실물 위치 반환 (태그 없음, 클라우드 없음)
핵심 AI	YOLO-World (객체 탐지) + CLIP (행동/이미지-텍스트 유사도)
실행 환경	로컬 PC (GPU 권장) – 엣지 컴퓨팅, 외부 전송 없음
인터페이스	Streamlit 웹 GUI (CLI 선택 가능)
언어/버전	Python 3.10+

1.1 핵심 동작 흐름

단계/모드	동작 내용
Step 1: Live Search	스마트폰/웹캠 스트림 → YOLO-World 30fps 탐지 → 즉시 위치 반환
Step 2: Quick Search	현재 화면에 없으면 → 과거 클립 CLIP 행동인식으로 역추적
모드 A: Zero-Shot	사전 등록 없이 텍스트 묘사만으로 탐색 (YOLO-World + CLIP)
모드 B: Few-Shot	3~5초 영상 촬영 → 특징 벡터 저장 → 벡터 유사도 정밀 탐색
이벤트 저장	MOG2 배경차감 → 움직임 후 3초 정지 감지 → 10~15초 MP4 클립 저장
프라이버시	모든 AI 추론을 로컬에서만 수행, 네트워크 전송 없음

2. 기술 스택 & 의존성 (Tech Stack)

2.1 핵심 패키지

패키지	설치 명령
ultralytics (YOLO-World)	pip install ultralytics>=8.1.0
openai-clip	pip install git+https://github.com/openai/CLIP.git
opencv-python	pip install opencv-python>=4.9.0
torch + torchvision	pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
streamlit	pip install streamlit>=1.32.0
faiss-cpu (벡터 DB)	pip install faiss-cpu
numpy, Pillow	pip install numpy pillow

📁 권장 디렉토리 구조

```
missingfind/ |— app/ | |— main.py          # Streamlit 진입점 | |— live_search.py    # Live
Search 모듈 | |— quick_search.py    # Quick Search 모듈 | |— event_capture.py    # MOG2
이벤트 감지 + 클립 저장 | |— vector_store.py    # FAISS 벡터 DB (Few-Shot) | |— utils.py
# 공통 유틸 |— models/ | |— (YOLO-World 가중치 자동 다운로드) |— data/ | |— clips/
# 저장된 MP4 이벤트 클립 | |— meta/          # JSON 메타데이터 | |— vectors/          #
Few-Shot 등록 벡터 |— tests/ | |— test_capture.py | |— test_live_search.py | |—
test_quick_search.py | |— test_vector_store.py |— requirements.txt |— config.yaml
```

3. 모듈별 구현 명세

3.1 이벤트 감지 & 클립 저장 (event_capture.py)

카메라 스트림을 MOG2 배경 차감으로 상시 모니터링하며, 물건이 놓이는 이벤트만 MP4 클립으로 저장합니다.

항목	명세
입력	웹캠/IP카메라 스트림 (cv2.VideoCapture)
출력	data/clips/{timestamp}_{cam_id}.mp4 + data/meta/{id}.json
이벤트 조건	전경 움직임 발생 후 3초 이상 정지 상태 유지
클립 길이	이벤트 전후 포함 10~15초
메타데이터	{ clip_id, timestamp, cam_id, bbox, confidence }
버퍼 전략	RollingBuffer 클래스: deque(maxlen=fps*pre_buffer_sec)

🗨 Copilot Prompt: event_capture.py 구현

```
# MissingFind - event_capture.py 구현 요청

다음 명세대로 Python 파일을 작성해줘.

## 목표
OpenCV MOG2 배경차감으로 카메라 스트림을 모니터링하고,
물건이 "내려놓아지는 이벤트"(움직임 후 3초 정지)를 감지하면
전후 10~15초 MP4 클립을 저장하는 EventCapture 클래스 구현.

## 클래스 인터페이스
class EventCapture:
    def __init__(self, cam_source: int|str, output_dir: str, config: dict)
    def start(self) -> None          # 백그라운드 스레드 시작
    def stop(self) -> None           # 스레드 정지
    def get_latest_clip(self) -> dict | None # 최신 클립 메타 반환

## 구현 요구사항
1. cv2.createBackgroundSubtractorMOG2() 사용
2. RollingBuffer: collections.deque(maxlen=fps * pre_buffer_sec) 로 프레임 사전 버퍼
3. 이벤트 감지 조건: 움직임 픽셀 > threshold 후 3초 이상 정지
4. 저장: fourcc='mp4v', data/clips/{cam_id}_{timestamp}.mp4
5. 메타데이터 JSON: clip_id(uuid4), timestamp(ISO), cam_id, event_bbox
6. 예외처리: 카메라 연결 실패 시 재시도(3회) 후 로깅
7. 스레드 안전: threading.Lock() 으로 deque 보호
8. config.yaml 에서 설정값 로드 (threshold, pre_buffer_sec 등)

## 금지사항
- 클라우드/외부 네트워크 전송 코드 금지
```

- 이벤트 없는 연속 녹화 금지 (스토리지 낭비 방지)

3.2 Live Search 모듈 (live_search.py)

YOLO-World를 사용해 실시간 카메라 프레임에서 자연어 쿼리와 일치하는 물체를 30fps 이상으로 탐지합니다.

항목	명세
입력	카메라 프레임 스트림 + 자연어 쿼리 문자열
출력	바운딩박스 좌표, 신뢰도 점수, 탐지 프레임 이미지
모델	YOLO-World (yolov8x-worldv2.pt, 자동 다운로드)
처리 속도	30fps 이상 목표 (GPU 환경)
쿼리 방식	model.set_classes([query]) 로 Open-Vocabulary 탐지
임계값	config.yaml: confidence_threshold (기본 0.35)

🗨 Copilot Prompt: live_search.py 구현

```
# MissingFind - live_search.py 구현 요청

## 목표
YOLO-World 모델로 실시간 카메라 스트림에서 자연어 쿼리 물체를 탐지하는
LiveSearch 클래스 구현.

## 클래스 인터페이스
class LiveSearch:
    def __init__(self, model_path: str, confidence: float = 0.35)
    def set_query(self, query: str) -> None
    def search_frame(self, frame: np.ndarray) -> list[dict]
        # 반환: [{ bbox:[x1,y1,x2,y2], confidence:float, label:str }]
    def annotate_frame(self, frame, results) -> np.ndarray
        # 바운딩박스 + 라벨 오버레이 이미지 반환

## 구현 요구사항
1. from ultralytics import YOLO 사용
2. model = YOLO("yolov8x-worldv2.pt") 로드 (최초 실행 시 자동 다운로드)
3. model.set_classes([query]) 로 Open-Vocabulary 쿼리 설정
4. 결과 없으면 빈 리스트 반환 (예외 아님)
5. annotate_frame: cv2.rectangle + cv2.putText 로 시각화
6. GPU/CPU 자동 감지: device = "cuda" if torch.cuda.is_available() else "cpu"
7. 모델은 싱글톤으로 한 번만 로드 (매 프레임마다 재로드 금지)

## 성능 요구사항
- frame 처리 시 불필요한 numpy 복사 최소화
```

- 결과 디렉터리에 timestamp 필드 포함 (time.time())

3.3 Quick Search 모듈 (quick_search.py)

저장된 MP4 클립에서 CLIP 유사도로 '물건을 내려놓는 장면'을 역추적합니다. 코사인 유사도로 상위 3~5개 후보 클립을 반환합니다.

항목	명세
입력	자연어 쿼리 + 클립 메타데이터 목록 (JSON)
출력	상위 N개 클립 경로 + 타임스탬프 + 유사도 점수 + 썸네일
CLIP 텍스트 프롬프트	"A person putting down {query} on a surface"
샘플링 레이트	클립당 1~2fps (연산 최적화)
유사도 계산	torch cosine_similarity (배치 처리)
정렬	유사도 내림차순, 상위 5개 반환

🗨 Copilot Prompt: quick_search.py 구현

```
# MissingFind - quick_search.py 구현 요청

## 목표
저장된 MP4 클립 목록에서 CLIP 모델로 특정 물건을 내려놓는 장면을
찾아 상위 후보를 반환하는 QuickSearch 클래스 구현.

## 클래스 인터페이스
class QuickSearch:
    def __init__(self, clips_dir: str, meta_dir: str, top_k: int = 5)
    def search(self, query: str) -> list[dict]
        # 반환: [{ clip_path, timestamp, score:float, thumbnail:np.ndarray }]

## 구현 요구사항
1. import clip; model, preprocess = clip.load("ViT-B/32")
2. 텍스트 프롬프트: f"A person putting down {query} on a table or floor"
3. 클립 프레임 샘플링: cv2.VideoCapture + 1fps 간격으로 추출
4. 배치 처리: 프레임을 64개씩 묶어 CLIP 이미지 인코더 통과
5. 코사인 유사도: torch.nn.functional.cosine_similarity 사용
6. 클립별 최고 점수 프레임을 대표 점수로 사용
7. 썸네일: 점수 가장 높은 프레임을 320x240으로 리사이즈
8. 메타 JSON에서 timestamp, cam_id 정보 함께 반환
9. 클립 파일 없으면 빈 리스트 반환 (예외 발생 금지)
10. @lru_cache 로 동일 쿼리 재검색 최적화
```

3.4 벡터 스토어 (vector_store.py) – Few-Shot 모드

앱에 등록된 물건의 CLIP 특징 벡터를 FAISS로 저장하고, 탐색 시 벡터 유사도로 정밀 검색합니다.

🗨 Copilot Prompt: vector_store.py 구현

```
# MissingFind - vector_store.py 구현 요청

## 목표
물건 등록 시 CLIP 벡터를 FAISS에 저장하고, 검색 시
코사인 유사도로 가장 가까운 벡터를 찾는 VectorStore 클래스.

## 클래스 인터페이스
class VectorStore:
    def __init__(self, store_dir: str)
    def register_item(self, item_name: str, video_path: str) -> str
        # 비디오에서 CLIP 벡터 추출 후 저장, item_id 반환
    def search(self, query_image: np.ndarray, top_k=3) -> list[dict]
        # 반환: [{ item_id, item_name, score }]
    def delete_item(self, item_id: str) -> bool
    def list_items(self) -> list[dict]

## 구현 요구사항
1. faiss.IndexFlatIP (내적=코사인 유사도, 벡터 정규화 후 사용)
2. 등록: 비디오 5fps 샘플링 → CLIP 임베딩 → 평균 벡터 → 정규화 저장
3. FAISS 인덱스를 store_dir/index.faiss 로 persist
4. 메타데이터(item_id↔item_name 매핑)는 store_dir/meta.json 저장
5. register_item 은 item_id = uuid4() 생성
6. 스레드 안전: threading.RLock 으로 read/write 보호
7. 인덱스가 없으면 자동 생성, 있으면 faiss.read_index 로 로드
```

3.5 Streamlit 메인 UI (main.py)

Streamlit 기반의 웹 대시보드로 라이브 피드, 이벤트 로그, 검색 결과를 실시간으로 표시합니다.

UI 구성요소	내용
사이드바	시작/중지 버튼, 카메라 설정, 감지 설정
라이브 카메라 탭	st.image()로 30fps 업데이트, 탐지 바운딩박스 오버레이
이벤트 로그 탭	저장된 클립 목록, 타임스탬프, 썸네일
검색 탭	텍스트 입력 → Live/Quick 자동 선택 → 결과 표시
물건 등록 탭	st.file_uploader or 카메라 녹화 → Few-Shot 등록
상태바	FPS, 클립 수, 스토리지 사용량

🗨 Copilot Prompt: main.py Streamlit UI 구현

```
# MissingFind - main.py Streamlit UI 구현 요청

## 목표
MissingFind 시스템의 Streamlit 웹 대시보드 구현.
모든 모듈(EventCapture, LiveSearch, QuickSearch, VectorStore)을 통합.

## UI 레이아웃
1. st.sidebar: 시작/중지 토글, 카메라 소스(0=웹캠), confidence 슬라이더
2. 상단 메트릭바: cols = st.columns(4) → [FPS, 이벤트 수, 클립 수, 스토리지GB]
3. st.tabs(["라이브 카메라", "이벤트 로그", "물건 검색", "물건 등록"])

## 검색 탭 동작 흐름
- st.text_input("찾을 물건을 입력하세요 (예: 검은 지갑)")
- 검색 버튼 클릭 시:
  1. LiveSearch.search_frame(current_frame) 먼저 시도
  2. 결과 없으면 QuickSearch.search(query) 실행
  3. 결과를 카드 형태로 st.columns 로 표시 (썸네일 + 시간 + 점수)

## 구현 요구사항
1. st.session_state 로 EventCapture 인스턴스 관리 (재실행 방지)
2. 라이브 피드: FRAME_PLACEHOLDER = st.empty() + while loop in thread
3. 이미지 표시: PIL.Image 변환 후 st.image(use_container_width=True)
4. 에러: st.error() 로 사용자 친화적 메시지 표시
5. config.yaml 로드: @st.cache_resource 데코레이터 사용
6. 물건 등록 탭: st.camera_input() 또는 st.file_uploader(.mp4) →
VectorStore.register_item()
```

4. 단계별 구현 플랜 (Implementation Plan)

🕒 전체 예상 기간

Phase 1~3 합계 약 2~3주 (집중 작업 기준). 각 Phase 완료 후 다음 Phase 진행 전 체크리스트 필수 확인.

Phase 1: 환경 구성 & 이벤트 캡처 (1~3 일)

1. Python 3.10+ 가상환경 생성: `python -m venv .venv`
2. `requirements.txt` 설치 및 GPU/CUDA 확인
3. `config.yaml` 작성 (카메라 소스, 경로, 임계값)
4. `event_capture.py` 구현 (Copilot 프롬프트 3.1 사용)
5. 테스트: `python tests/test_capture.py`

Phase 2: AI 검색 모듈 (3~7 일)

6. `live_search.py` 구현 (Copilot 프롬프트 3.2 사용)
7. YOLO-World 모델 다운로드 확인 및 단위 테스트
8. `quick_search.py` 구현 (Copilot 프롬프트 3.3 사용)
9. CLIP 설치 확인 및 단위 테스트
10. `vector_store.py` 구현 (Copilot 프롬프트 3.4 사용)

Phase 3: UI 통합 & 최적화 (2~4 일)

11. `main.py` Streamlit UI 구현 (Copilot 프롬프트 3.5 사용)
12. 모듈 통합 테스트 (전체 파이프라인)
13. 성능 최적화: FPS 측정, 배치 크기 조정
14. 예외처리 강화 및 로깅 구성
15. `streamlit run app/main.py` 로 최종 확인

5. 구현 체크리스트

5.1 환경 구성

	항목	검증 내용	담당/우선순위
☐	Python 버전	python --version → 3.10+ 확인	필수 / High
☐	CUDA 확인	torch.cuda.is_available() → True (GPU 환경)	권장 / High
☐	패키지 설치	pip install -r requirements.txt 에러 없음	필수 / High
☐	YOLO-World 다운로드	yolov8x-worldv2.pt 자동 다운로드 확인	필수 / High
☐	CLIP 설치	import clip; clip.load('ViT-B/32') 에러 없음	필수 / High
☐	디렉토리 생성	data/clips/, data/meta/, data/vectors/ 폴더 존재	필수 / Med
☐	config.yaml	cam_source, output_dir, threshold 값 설정 완료	필수 / Med

5.2 이벤트 캡처 모듈

	항목	검증 내용	담당/우선순위
☐	카메라 연결	cv2.VideoCapture(0) 정상 작동 확인	필수 / High
☐	이벤트 감지	물건 내려놓을 때 콘솔에 'Event detected' 출력	필수 / High
☐	클립 저장	data/clips/에 .mp4 파일 생성 확인	필수 / High
☐	메타데이터	data/meta/에 JSON 파일 생성 및 필드 확인	필수 / High
☐	빈방 저장 방지	움직임 없을 때 파일 생성 안됨 확인	필수 / Med
☐	재시도 로직	카메라 연결 실패 시 3회 재시도 후 에러 로그	권장 / Med
☐	스레드 안전	동시 접근 시 Lock 정상 작동 (pytest 병렬)	권장 / Low

5.3 Live Search 모듈

	항목	검증 내용	담당/우선순위
☐	모델 로드	LiveSearch() 초기화 시 1회만 로드 확인 (로그)	필수 / High
☐	Zero-Shot 탐지	'열쇠', '지갑' 등 텍스트 쿼리로 탐지 성공	필수 / High
☐	바운딩박스	annotate_frame() 결과 이미지에 박스/라벨 표시	필수 / High
☐	FPS 측정	time.time() 기반 FPS ≥ 15 (GPU), ≥ 5 (CPU)	필수 / High

☐	빈 결과 처리	물건 없는 프레임에서 빈 리스트 반환 (예외 없음)	필수 / Med
☐	쿼리 변경	set_query() 호출 후 즉시 새 쿼리 적용 확인	필수 / Med

5.4 Quick Search 모듈

	항목	검증 내용	담당/우선순위
☐	클립 샘플링	1fps로 프레임 추출 후 CLIP 인코더 통과 확인	필수 / High
☐	유사도 점수	search() 반환값 score 범위 0~1 확인	필수 / High
☐	상위 K 정렬	top_k=5 시 점수 내림차순 정렬 확인	필수 / High
☐	썸네일 반환	thumbnail 필드가 320x240 np.ndarray 확인	필수 / Med
☐	빈 클립 처리	clips_dir 비어있을 때 빈 리스트 반환	필수 / Med
☐	캐시 작동	@lru_cache 동일 쿼리 재검색 속도 향상 확인	권장 / Low

5.5 벡터 스토어

	항목	검증 내용	담당/우선순위
☐	등록 성공	register_item() 후 list_items()에 항목 나타남	필수 / High
☐	FAISS 영속성	프로그램 재시작 후 index.faiss 로드 및 검색 성공	필수 / High
☐	벡터 정규화	저장 벡터 L2 norm ≈ 1.0 확인 (np.linalg.norm)	필수 / Med
☐	삭제 기능	delete_item() 후 검색 결과에서 제외 확인	필수 / Med
☐	스레드 안전	동시 등록/검색 시 RLock 정상 작동	권장 / Low

5.6 Streamlit UI

	항목	검증 내용	담당/우선순위
☐	앱 실행	streamlit run app/main.py 에러 없이 실행	필수 / High
☐	라이브 피드	웹캠 화면이 대시보드에 실시간 표시	필수 / High
☐	검색 플로우	텍스트 입력 → 검색 → 결과 카드 표시	필수 / High
☐	이벤트 로그	클립 저장 시 이벤트 로그 탭 자동 갱신	필수 / High

☐	물건 등록	카메라 촬영/파일 업로드 → 등록 성공 메시지	필수 / Med
☐	에러 표시	카메라 없을 때 <code>st.error()</code> 메시지 표시	필수 / Med
☐	상태 유지	페이지 새로고침 시 <code>session_state</code> 유지	권장 / Low

6. 디버그 & 테스트 가이드

6.1 단위 테스트 Copilot 프롬프트

🗨 Copilot Prompt: tests/test_capture.py 작성

```
# test_capture.py 작성 요청

EventCapture 클래스의 단위 테스트를 pytest로 작성해줘.

## 테스트 케이스
1. test_camera_open: 카메라 소스 0으로 VideoCapture 열기 성공
2. test_event_detection: 합성 움직임 프레임 주입 시 이벤트 감지
3. test_clip_saved: 이벤트 감지 후 mp4 파일 생성 확인
4. test_meta_saved: 이벤트 감지 후 JSON 메타 파일 생성 및 필드 확인
5. test_no_clip_static: 정적 프레임 연속 입력 시 클립 미생성 확인
6. test_thread_safety: 동시 get_latest_clip() 호출 시 예외 없음

## Mock 요구사항
- cv2.VideoCapture를 unittest.mock.patch로 모킹
- 합성 프레임: np.zeros((480,640,3), dtype=np.uint8) 기반
- 움직임 프레임: 일부 픽셀을 255로 설정
- tmp_path fixture로 임시 디렉토리 사용
```

🗨 Copilot Prompt: tests/test_live_search.py 작성

```
# test_live_search.py 작성 요청

LiveSearch 클래스의 단위 테스트를 pytest로 작성해줘.

## 테스트 케이스
1. test_model_loads_once: LiveSearch() 두 번 호출해도 모델 1회 로드 확인
2. test_search_returns_list: search_frame(frame) 반환값이 list 타입 확인
3. test_bbox_format: 결과 딕셔너리에 bbox, confidence, label 키 존재 확인
4. test_empty_frame_no_exception: 검은 프레임에서 예외 없이 빈 리스트 반환
5. test_annotate_frame_shape: annotate_frame() 출력 shape == 입력 shape
6. test_set_query: set_query("지갑") 후 model.set_classes 호출 확인

## Mock 요구사항
- YOLO 모델 자체는 mock (실제 GPU 불필요한 테스트 환경 고려)
- conftest.py에 sample_frame fixture (640x480 랜덤 이미지) 정의
```

6.2 통합 테스트 시나리오

시나리오	검증 내용
시나리오 1: 전체 파이프라인	EventCapture 시작 → 물건 내려놓기 → 클립 저장 → QuickSearch('열쇠') → 해당 클립 상위 반환 확인

시나리오 2: Live 우선 탐색	LiveSearch 먼저 실행 → 발견 시 QuickSearch 미실행 확인
시나리오 3: Few-Shot 정밀 탐색	VectorStore 등록 → 동일 물건 카메라 비춤 → score > 0.8 확인
시나리오 4: 빈 클립 폴더	clips_dir 빈 상태에서 QuickSearch → 빈 리스트 반환, 크래시 없음
시나리오 5: 카메라 미연결	VideoCapture 실패 → st.error 메시지 표시, 앱 종료 없음

6.3 주요 디버그 포인트 & 해결법

문제	해결 방법
YOLO-World 로드 느림	첫 실행 시 모델 다운로드 (수백MB). @st.cache_resource 로 캐싱 필수
CLIP import 실패	pip install git+https://github.com/openai/CLIP.git 재설치 확인
FPS 저하 (CPU 환경)	CLIP 샘플링을 0.5fps로 낮추거나, 배치 크기 32로 줄이기
MOG2 오탐 (조명 변화)	config.yaml: history=200, varThreshold=50으로 조정
FAISS 인덱스 손상	data/vectors/ 삭제 후 재등록 (VectorStore 초기화)
Streamlit 화면 멈춤	라이브 피드 스레드를 daemon=True로 설정했는지 확인
클립 저장 경로 오류	os.makedirs(output_dir, exist_ok=True) 코드 추가 여부 확인

6.4 성능 벤치마크 Copilot 프롬프트

🗨️ Copilot Prompt: benchmark.py 작성

```
# benchmark.py 작성 요청

MissingFind 각 모듈의 처리 속도를 측정하는 벤치마크 스크립트 작성.

## 측정 항목
1. LiveSearch: 단일 프레임 처리 시간 (ms) + FPS 계산 (100프레임 평균)
2. QuickSearch: 10개 클립 검색 총 소요 시간 (초)
3. EventCapture: 이벤트 감지 반응 지연 시간 (ms)
4. VectorStore: 등록 10개 + 검색 100회 평균 시간

## 출력 형식
=== MissingFind Benchmark ===
[LiveSearch]    avg: 28.3ms/frame | 35.3 FPS
[QuickSearch]   10 clips: 4.2s total | 0.42s/clip
[EventCapture]  detection latency: 312ms avg
[VectorStore]   register: 1.2s/item | search: 3.1ms/query

## 요구사항
```

- `timeit.repeat(number=10, repeat=3)` 사용
- GPU/CPU 정보도 출력: `torch.cuda.get_device_name(0)`
- 결과를 `benchmark_result.json` 파일로 저장

7. config.yaml 전체 명세

 이 파일을 복사해 프로젝트 루트에 config.yaml로 저장하세요.

```
camera: source: 0          # 웹캠 인덱스 (IP카메라: "rtsp://...") fps: 30 resolution: [1280, 720]
capture: pre_buffer_sec: 5  # 이벤트 전 버퍼 (초) post_buffer_sec: 10      # 이벤트 후 녹화
      (초) motion_threshold: 500 # MOG2 전경 픽셀 수 임계값 stillness_sec: 3.0      # 정지 판단
      시간 (초) output_dir: "data/clips" meta_dir: "data/meta" live_search: model: "yolov8x-worldv2.pt"
confidence: 0.35 device: "auto"      # "auto" | "cuda" | "cpu" quick_search: sample_fps: 1
# 클립 샘플링 속도 batch_size: 64 top_k: 5 clip_model: "ViT-B/32" vector_store: store_dir:
"data/vectors" register_fps: 5      # 등록 시 프레임 샘플링 ui: page_title: "MissingFind"
theme: "dark"
```

8. 빠른 시작 가이드 (Quick Start)

8.1 Copilot 사용 팁

 Copilot을 가장 잘 활용하는 방법

1. 각 섹션의 프롬프트 박스 내용을 그대로 복사해 Copilot Chat에 붙여넣기 2. 파일 상단에 # [파일명] 주석과 핵심 import를 먼저 작성하면 Copilot이 맥락을 더 잘 파악 3. 구현 후 즉시 해당 체크리스트 항목을 확인하고, 실패 항목은 디버그 섹션 참조 4. 한 번에 전체 파일보다 클래스/함수 단위로 요청하면 품질이 더 높음

8.2 실행 명령어

```
# 1. 가상환경 활성화
python -m venv .venv && source .venv/bin/activate #
Windows: .venv\Scripts\activate

# 2. 패키지 설치
pip install -r requirements.txt
pip install git+https://github.com/openai/CLIP.git

# 3. 테스트 실행
pytest tests/ -v

# 4. 벤치마크
python benchmark.py

# 5. 앱 실행
streamlit run app/main.py
```

☒ 완료 기준

모든 pytest 테스트 통과 + Streamlit 앱에서 텍스트 질의 시 Live/Quick Search 결과가 화면에 표시되면 Phase 3 완료입니다. 이후 IP카메라 연동, 모바일 클라이언트 연결, 음성 입력 추가 등으로 확장할 수 있습니다.